

Salesforce Data Migration Toolkit

End-to-End Schema & Data Migration



Agenda

Schema Creation & Deployment

Load Sequencing Strategy

Load Plan Generation

Data Population using Bulk APIs

End-to-End Migration using SMT Studio

Salesforce Schema Builder

Tool: sf-schema-builder

<https://www.npmjs.com/package/sf-schema-builder>

Purpose

Build and manage **Salesforce metadata schema**

Create objects, fields, and relationships

Prepare **source org structure** for migration

Steps

1 Create SFDX Project

```
sf project generate -n projectName
```

2 Navigate to Project

```
cd projectName
```

3 Retrieve Metadata from Source Org

```
sf project retrieve start -o source0rg -x package.xml -w 20
```

4 Deploy Metadata to Target Org

```
sf project deploy start -o targetOrg -x package.xml -w 20
```

Load Sequencer App

Tool: sf-load-sequencer-ui

<https://www.npmjs.com/package/sf-load-sequencer-ui>

Purpose

Determine **correct object load order**

Avoid **lookup and dependency failures**

Visualize object relationships

Key Concept

👉 Objects must be loaded in dependency order:

- Parent objects first

- Child objects later

✓ Output

Ordered list of objects

Use this order in **Load Plan JSON**



Load Plan Generator CLI

Tool: sf-load-plan-generator

<https://www.npmjs.com/package/sf-load-plan-generator>

Purpose

Generate **load plan JSON per object**

Define:

Queries

Field mappings

Keys

Workflow

Generate load plan for **each object**

Combine into a **single JSON** array

Follow order from **Load Sequencer**



Tool: sf-utils-cli

<https://www.npmjs.com/package/sf-utils-cli>

Purpose

Perform **data upload** using **Bulk API 2.0**

Efficient for **large datasets**

Features

CSV-based data upload

High-performance bulk processing

Error handling & retry support

Usage

Prepare input CSV files

Upload using Bulk API commands

Validate results

Salesforce Migration Tool (SMT) Studio

Tool: `smt-studio`

<https://www.npmjs.com/package/smt-studio>

Purpose

End-to-end data migration between orgs

Uses Load Plan JSON

What is Load Plan?

Defines:

- Objects

- Queries

- Field mappings

- Relationships

Example Load Plan

```
[
  {
    "object": "Manufacturer__c",
    "compositeKeys": ["Global_Key__c"],
    "query": "SELECT Global_Key__c, Name, Country__c FROM Manufacturer__c",
    "fieldMappings": {
      "Global_Key__c": "Global_Key__c",
      "Name": "Name",
      "Country__c": "Country__c"
    }
  },
  {
    "object": "Vehicle__c",
    "compositeKeys": ["Global_Key__c"],
    "query": "SELECT Global_Key__c, Name, Model__c, Manufacturer__r.Global_Key__c FROM Vehicle__c",
    "fieldMappings": {
      "Global_Key__c": "Global_Key__c",
      "Name": "Name",
      "Model__c": "Model__c",
      "Manufacturer__c": {
        "lookup": {
          "object": "Manufacturer__c",
          "key": "Global_Key__c",
          "field": "Manufacturer__r.Global_Key__c"
        }
      }
    }
  }
]
```

Lookup Handling

Uses **reference resolution**

Maps parent-child relationships using:

Object

Key

Field

End-to-End Workflow

 Build schema using Schema Builder

 Retrieve & deploy metadata via SFDX

 Determine load order using Sequencer

 Generate load plans per object

 Upload data via SF Utils (Bulk API)

 Execute migration via SMT Studio



Best Practices

Use **Global Keys** for data integrity

Validate **SOQL queries** before execution

Always follow **dependency order**

Test in **lower environments first**

Maintain **version-controlled load plans**

❑ Conclusion

These tools together provide a **complete migration ecosystem**

Modular, scalable, and automation-friendly

Enables **reliable Salesforce data migration**

🙌 **Thank You**

Questions?